CS216 - Assignment 2

Team name : Malicious_Nodes
Members:

Name	Roll No.
S Varshith Reddy	240001071
Harshith Pasupuleti	240003034
Trijal Mathuria	240001073
Sujith D	240001028

GitHub Repository:
https://github.com/Yagami-light45/CS216-Malicious_Nodes-Bitcoin_Transaction_Lab

Objective :
The objective of this assignment is to understand the mechanics of creating and validating Bitcoin transactions using two different address formats: Legacy (P2PKH) and SegWit (P2SH-P2WPKH).

Through this lab, we developed programmatic scripts to interact directly with a local bitcoind node running in regression test (regtest) mode. The tasks accomplished include:

1. **Legacy Transactions (Part 1):** Generating P2PKH addresses, funding them, and executing a chain of transactions (A -> B -> C) while extracting and debugging the cryptographic unlocking and locking scripts (scriptSig and scriptPubKey).
2. **SegWit Transactions (Part 2):** Generating P2SH-SegWit addresses, executing a similar transaction chain (A -> B -> C), and analyzing the structural changes in the scripts.

3. **Comparative Analysis (Part 3):** Comparing the transaction structures, evaluating the reduction in physical size versus virtual bytes (vbytes) introduced by the Segregated Witness upgrade, and outlining the benefits of these size reductions.

Legacy Address Transactions (P2PKH)

Workflow Explanation

- **Network Setup:** Initialize a local Bitcoin node in regtest mode to create an isolated testing environment with standard fee configurations.
- **Address Creation & Funding:** Generate three standard Legacy addresses (A, B, and C). Fund Address A by mining blocks directly to it, providing the initial Unspent Transaction Output (UTXO) needed to start the chain.
- **First Transfer (A ->B):** Construct a raw transaction to send funds from Address A to Address B. Sign the transaction using A's wallet and broadcast it to the local network.
- **Second Transfer (B -> C):** Locate the newly created UTXO sitting at Address B. Use this specific UTXO as the input to build, sign, and broadcast a second raw transaction sending funds from Address B to Address C.
- **Script Analysis:** Decode both confirmed transactions to extract their raw metrics (Size, Vsize, Weight). Finally, isolate the locking (scriptPubKey) and unlocking (scriptSig) scripts and run them through the btcdeb debugger to verify how the legacy network processes public keys and signatures.

1. Transaction Workflow & UTXO Management

- **Funding Address A**

Address A was funded with an initial balance of 5.0 BTC using the `sendtoaddress` RPC command. This created the initial Unspent Transaction Output (UTXO) required to begin the transaction chain.

```
======================================================================
 PART 2: FUNDING ADDRESS A
======================================================================
Starting wallet balance: 1149.99821630 BTC

Sending 5.0 BTC to Address A (mw63FzSKzSPo6BaCu6ijBx43WDiUQoB1Aw)...
Funding TXID: ac8b87cfb7eeb7c77e2d854bcf9ec1be8754c6cf05eb9b62de8a5489ad82bdca

Mining 1 block to confirm the transaction...
Confirmations: 1
```

- **Transaction A ->B**

Input: The UTXO from Address A.
Outputs: 2.0 BTC sent to Address B , with the remaining balance minus fees sent back to Address A as change.

TXID:`"bf23ac6ed27f08e9937790361fd80a37db219036d8b72e4082119ef2120b3eb7"`


```
{
  "txid": "bf23ac6ed27f08e9937790361fd80a37db219036d8b72e4082119ef2120b3eb7",
  "from": "Address A",
  "to": "Address B",
  "amount": 2.0,
  "decoded_transaction": {
    "txid": "bf23ac6ed27f08e9937790361fd80a37db219036d8b72e4082119ef2120b3eb7",
    "hash": "bf23ac6ed27f08e9937790361fd80a37db219036d8b72e4082119ef2120b3eb7",
    "version": 2,
    "size": 225,
    "vsize": 225,
    "weight": 900,
    "locktime": 0,
    "vin": [
      {
        "txid": "ac8b87cfb7eeb7c77e2d854bcf9ec1be8754c6cf05eb9b62de8a5489ad82bdca",
        "vout": 1,
        "scriptSig": {
          "asm": "304402203076f520f284ab982d67545fba74a91420b89d23a16c3422c984e121a0540f7c022011c8a5d40a3fcbf8c383f2bc27436a76d8ea8e5f9ab686484e7e89a67aaf9e61[ALL] 03822c5813323ca457f8d1da1d18f98ddabb4177db70aa96c5618c5fa64924e558",
          "hex": "473044022030 76f520f284ab982d67545fba74a91420b89d23a16c3422c984e121a0540f7c022011c8a5d40a3fcbf8c383f2bc27436a76d8ea8e5f9ab686484e7e89a67aaf9e6101210382 2c5813323ca457f8d1da1d18f98ddabb4177db70aa96c5618c5fa64924e558"
        },
        "sequence": 4294967293
      }
    ],
    "vout": [
      {
        "value": "2.00000000",
        "n": 0,
        "scriptPubKey": {
          "asm": "OP_DUP OP_HASH160 6c89d116dc78c177b1788ec1ce16b0c7951f0920 OP_EQUALVERIFY OP_CHECKSIG",
          "desc": "addr(mqQrNSvPh81SRCvW78uVxenp1NPBHPc8wz)#6vvmg4c4",
          "hex": "76a9146c89d116dc78c177b1788ec1ce16b0c7951f092088ac",
          "address": "mqQrNSvPh81SRCvW78uVxenp1NPBHPc8wz",
          "type": "pubkeyhash"
        }
      },
      {
        "value": "2.99990000",
        "n": 1,
        "scriptPubKey": {
          "asm": "OP_DUP OP_HASH160 aacbc1f1c2fb9202444f61712cd34a09767b0a3a OP_EQUALVERIFY OP_CHECKSIG",
          "desc": "addr(mw63FzSKzSPo6BaCu6ijBx43WDiUQo81Aw)#vs29x9p1",
          "hex": "76a914aacbc1f1c2fb9202444f61712cd34a09767b0a3a88ac",
          "address": "mw63FzSKzSPo6BaCu6ijBx43WDiUQo81Aw",
          "type": "pubkeyhash"
        }
      }
    ]
  }
}
```


- **Transaction B ->C**

Input: To execute this transfer, the specific `A -> B` transaction was used as the input UTXO. This proves the continuous flow of funds.
Outputs: 1.0 BTC sent to Address C, with the remaining balance minus fees sent back to Address B as change.

TXID:`"81b558aabf099984c64c46e43475323b3aad7b2390766ed7f247e425b1709792"`

```
"txid": "81b558aabf099984c64c46e43475323b3aad7b2390766ed7f247e425b1709792",
"from": "Address B",
"to": "Address C",
"amount": 1.0,
"decoded_transaction": {
  "txid": "81b558aabf099984c64c46e43475323b3aad7b2390766ed7f247e425b1709792",
  "hash": "81b558aabf099984c64c46e43475323b3aad7b2390766ed7f247e425b1709792",
  "version": 2,
  "size": 225,
  "vsize": 225,
  "weight": 900,
  "locktime": 0,
  "vin": [
    {
      "txid": "bf23ac6ed27f08e9937790361fd80a37db219036d8b72e4082119ef2120b3eb7",
      "vout": 0,
      "scriptSig": {
        "asm": "304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda2990ff988c75023e2bc4819a6923a4ec[ALL] 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e",
        "hex": "47304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda2990ff988c75023e2bc4819a6923a4ec0121027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e"
      },
      "sequence": 4294967293
    }
  ],
  "vout": [
    {
      "value": "1.00000000",
      "n": 0,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 cffee37637662a96fbb507e9aafb259c9bd97baa OP_EQUALVERIFY OP_CHECKSIG",
        "desc": "addr(mzUjWjB4Fr5ePT2jKUvgfE5LDLwdgVErfc)#1jp743ew",
        "hex": "76a914cffee37637662a96fbb507e9aafb259c9bd97baa88ac",
        "address": "mzUjWjB4Fr5ePT2jKUvgfE5LDLwdgVErfc",
        "type": "pubkeyhash"
      }
    },
    {
      "value": "0.99990000",
      "n": 1,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 6c89d116dc78c177b1788ec1ce16b0c7951f0920 OP_EQUALVERIFY OP_CHECKSIG",
        "desc": "addr(mqQrNSvPh81SRCvW78uVxenp1NPBHPc8wz)#6vvmg4c4",
        "hex": "76a9146c89d116dc78c177b1788ec1ce16b0c7951f092088ac",
        "address": "mqQrNSvPh81SRCvW78uVxenp1NPBHPc8wz",
        "type": "pubkeyhash"
      }
    }
  ]
},
"btcdeb_command": "btcdeb '47304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda2990ff988c75023e2bc4819a6923a4ec0121027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01c
```


3. Script Validation Explanation

In a Legacy P2PKH transaction, the unlocking script (`scriptSig` / response) provides the user's cryptographic `<Signature>` and `<Public Key>`.
The locking script (`scriptPubKey` / challenge) validates this by running `OP_DUP` to duplicate the public key, and `OP_HASH160` to hash it. It then pushes the destination address hash onto the stack and uses `OP_EQUALVERIFY` to ensure the hashes match. Finally, `OP_CHECKSIG` consumes the signature and public key to verify the mathematical proof of ownership.

4. Debugging Transaction B -> C

Due to a known issue affecting **btcdeb version 5.0 when running on Ubuntu 24.04**, the debugger consistently terminated with a **“core dumped / buffer overflow”** error during execution on our testing machine. To avoid this environment-specific issue, the debugging step was executed on a **different system running an earlier Ubuntu version**. The following output shows a successful demonstration of the script execution and stack trace produced during the debugging process.

```
"btcdeb_command": "btcdeb
'47304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda2990
ff988c75023e2bc4819a6923a4ec0121027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e76a9146c89d116dc7
8c177b1788ec1ce16b0c7951f092088ac'"
```

INITIAL STATE :

```
sairaj@sairaj:~$ btcdeb '47304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda29
90ff988c75023e2bc4819a6923a4ec0121027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e76a9146c89d116dc78c177b1788ec1ce16b0c
7951f092088ac'
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
7 op script loaded. type `help` for usage information
script                                                                  |  stack
------------------------------------------------------------------------+--------
304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939... |
027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e |
OP_DUP                                                                  |
OP_HASH160                                                              |
6c89d116dc78c177b1788ec1ce16b0c7951f0920                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0000 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fdbda2990ff988c75023e2bc4819
a6923a4ec01
```

MID EXECUTION :

```
btcdeb> step
                <> PUSH stack 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a29390babfb00102202c9a986840570fbebb65aa08656fd
a2990ff988c75023e2bc4819a6923a4ec01
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...
OP_DUP                                                                  |
OP_HASH160                                                              |
6c89d116dc78c177b1788ec1ce16b0c7951f0920                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0001 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
btcdeb> step
                <> PUSH stack 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_DUP                                                                  | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
OP_HASH160                                                              | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...
6c89d116dc78c177b1788ec1ce16b0c7951f0920                                |
OP_EQUALVERIFY                                                          |
OP_CHECKSIG                                                             |
#0002 OP_DUP
btcdeb> step
                <> PUSH stack 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_HASH160                                                              | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
6c89d116dc78c177b1788ec1ce16b0c7951f0920                                | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
OP_EQUALVERIFY                                                          | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...
OP_CHECKSIG                                                             |
#0003 OP_HASH160
btcdeb> step
                <> POP  stack
                <> PUSH stack 6c89d116dc78c177b1788ec1ce16b0c7951f0920
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
6c89d116dc78c177b1788ec1ce16b0c7951f0920                                |                                 6c89d116dc78c177b1788ec1ce16b0c7951f0920
OP_EQUALVERIFY                                                          | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
OP_CHECKSIG                                                             | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...
#0004 6c89d116dc78c177b1788ec1ce16b0c7951f0920
btcdeb> step
                <> PUSH stack 6c89d116dc78c177b1788ec1ce16b0c7951f0920
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
OP_EQUALVERIFY                                                          |                                 6c89d116dc78c177b1788ec1ce16b0c7951f0920
OP_CHECKSIG                                                             |                                 6c89d116dc78c177b1788ec1ce16b0c7951f0920
                                                                        | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
                                                                        | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...
#0005 OP_EQUALVERIFY
```

FINAL STATE :

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
OP_CHECKSIG                                                     | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
                                                                | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...

#0006 OP_CHECKSIG
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
error: Signature is found in scriptCode
btcdeb> step
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
                                                                | 027a6d055097ed2006c5930a358dfee054178a3f5992ca9dda8de8fc01ce1dd52e
                                                                | 304402205ff58b39f193afa1e036683ae8f2c85fadc013c2e314862a66a2939...

#0006 OP_CHECKSIG
```

Segwit Report

Workflow Explanation (Part 2 – SegWit P2SH-P2WPKH)

Network Setup:
A local Bitcoin node is initialized in **regtest mode** and accessed through RPC using the `bitcoinrpc` library, providing a controlled environment to test SegWit transactions.

Address Creation & Funding:
A wallet named **lab** is created or loaded, and three **P2SH-wrapped SegWit addresses** (A′, B′, C′) are generated. Address **A′** is funded by mining blocks to create a spendable UTXO.

First Transfer (A′ → B′):
A raw transaction is constructed to send **2 BTC from A′ to B′**, signed with the wallet, broadcast to the network, and confirmed by mining a block.

Second Transfer (B′ → C′):
The UTXO created at **B′** is used as the input for a second transaction sending **1 BTC from B′ to C′**, again signed and confirmed through mining.

Script & Witness Analysis:
Both transactions are decoded to extract their **witness data, scriptSig, and scriptPubKey**. The **signature and public key stored in the witness stack** are analyzed along with the locking script to understand how SegWit validates transactions.

Bitcoin Script Debugging:
The extracted signature, public key, and script hash are used to generate a **btcdeb debugging command**, allowing step-by-step execution of the script to observe the stack operations involved in SegWit transaction validation.

Transaction Metrics:
Finally, the **size, virtual size (vsize), and weight units** of both transactions are recorded to illustrate how SegWit improves transaction efficiency.

Step-1 : Start up of bitcoin daemon and checking the status of connection.

```
varshith@varshith: ~/.bitcoin
varshith@varshith:~/.bitcoin$ bitcoind -regtest -daemon
Bitcoin Core starting
varshith@varshith:~/.bitcoin$ bitcoin-cli getblockchaininfo
{
  "chain": "regtest",
  "blocks": 925,
  "headers": 925,
  "bestblockhash": "656f28ab766ce55e3ddd819898307515dbbfbba2a4c3d40b3162809d6562
62c5",
  "difficulty": 4.656542373906925e-10,
  "time": 1772970765,
  "mediantime": 1772970764,
  "verificationprogress": 1,
  "initialblockdownload": false,
  "chainwork": "0000000000000000000000000000000000000000000000000000000000000073c
",
  "size_on_disk": 281900,
  "pruned": false,
  "warnings": ""
}
varshith@varshith:~/.bitcoin$
```

Step-2 : Creating addresses and funding and creating transactions from A to B and B to C

```
SegWit P2SH-P2WPKH Transactions
Connected to: regtest
Address A: 2MxgVrYi8coPBUmaNwP6heAJyCydBfZkiey
Address B: 2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs
Address C: 2NDzbuBLSwnQQV7F9MDXaXXr25g7BLWKSiY

Mining 101 blocks to A

Creating Transaction A -> B (2 BTC)
TXID: ec59ddc4f77ef85ec6925e4745976d56ad97270db9f97dffa5ba470dcc326474

Creating Transaction B -> C (1 BTC)
TXID: ded13b0b763b5b62c15d9448ab850de8c8cad0be24d4fd2eacdc7b919809fe62
```

The generated addresses are P2SH-SegWit (P2SH-P2WPKH) addresses.These addresses wrap a SegWit witness program inside a P2SH script.(Sample saved transaction is shown below).

```
"signed_tx": {
  "txid": "ec59ddc4f77ef85ec6925e4745976d56ad97270db9f97dffa5ba470dcc326474",
  "hash": "0e472f3ab4a7bd0eef60180c6e205e86c129727fc2710cfeebd049da84fde904",
  "version": 2,
  "size": 247,
  "vsize": 166,
  "weight": 661,
  "locktime": 0,
  "vin": [
    {
      "txid": "7fa77a9bd92713203a36b1cdb133d92f1469cb5bc19c36ca33c0bed19c25e584",
      "vout": 0,
      "scriptSig": {
        "asm": "00149a29c21176818ab0bbe306699ed70aacccf57ceb",
        "hex": "1600149a29c21176818ab0bbe306699ed70aacccf57ceb"
      },
      "txinwitness": [
        "304402203c4137b3bff3a670ac7f71d4dbd31e8badc6a43b35d9a6dfc5c4853262794c5202207811b1b8d
        1be572e41a61560ed86ceed0cd823d473ff6770e7257e23983114dc01",
        "02aa04558b7662aefd5073d10969e38ea33bdfffbe0df3dbc4c841d14b31ad849e"
      ],
      "sequence": 4294967293
    }
  ],
  "vout": [
    {
      "value": "2.00000000",
      "n": 0,
      "scriptPubKey": {
        "asm": "OP_HASH160 045cd58f134b43d83b9c19ea231fb23df2fc98b5 OP_EQUAL",
        "desc": "addr(2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs)#k7k62l4z",
        "hex": "a914045cd58f134b43d83b9c19ea231fb23df2fc98b587",
        "address": "2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs",
        "type": "scripthash"
      }
    },
```


The UTXO from A'->B' is used as input for B'->C'.

Step-3 Script Validation Explanation

In a **SegWit P2SH-P2WPKH transaction**, the unlocking data is moved to the **witness field**, which contains the user's cryptographic `<Signature>` and `<Public Key>`. The `scriptSig` only holds the **SegWit redeem script**, which references the witness program.

The locking script (scriptPubKey / challenge) checks the ownership of the funds by hashing the provided public key using **OP_HASH160** and comparing it with the stored public key hash. If the hashes match, the

script proceeds to **OP_CHECKSIG**, which verifies that the signature corresponds to the provided public key.

If the signature is valid and the hashes match, the script evaluation succeeds, proving that the spender has the correct private key and therefore the authority to spend the transaction output.

Step-4: BTCDEB command

```
Run this command for btcdeb debugging:

btcdeb '[304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce622275fe76b36c90a23b4c8f6b77ff3679b38b03def
cbb801 029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473]' 'OP_DUP OP_HASH160 444a86b3439c8b23c9b53a22fd19345de2dc53ae OP_EQUALV
ERIFY OP_CHECKSIG'
```

btcdeb '[signature pubkey]'
'OP_DUP OP_HASH160 ... OP_EQUALVERIFY OP_CHECKSIG'
The above command shows a structure of the btcdeb command generated by the code to test the given transaction.

The btcdeb command is used to debug and validate the execution of Bitcoin scripts. It allows step-by-step verification of the scriptSig, scriptPubKey and witness stack during transaction validation.


```
{
  "signature":
  "304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce
  622275fe76b36c90a23b4c8f6b77ff3679b38b03defcbb801",
  "pubkey": "029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473",
  "implicit_segwit_script": "OP_DUP OP_HASH160 444a86b3439c8b23c9b53a22fd19345de2dc53ae
  OP_EQUALVERIFY OP_CHECKSIG",
  "command": "btcdeb '
  [304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce
  622275fe76b36c90a23b4c8f6b77ff3679b38b03defcbb801
  029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473]' 'OP_DUP OP_HASH160
  444a86b3439c8b23c9b53a22fd19345de2dc53ae OP_EQUALVERIFY OP_CHECKSIG'"
}
```


- We have also added a script to save the commands into a simple json file in the code base.

Step-5: Running the btcdeb command in terminal

Due to a known issue affecting **btcdeb version 5.0 when running on Ubuntu 24.04**, the debugger consistently terminated with a **“core dumped / buffer overflow”** error during execution on our testing machine. This behavior corresponds to a bug reported in the btcdeb [GitHub](#) issues related to memory handling in certain environments.

To avoid this environment-specific issue, the debugging step was executed on a **different system running an earlier Ubuntu version**, where btcdeb functioned correctly. The following output shows a successful demonstration of the script execution and stack trace produced during the debugging process.

```
sairaj@sairaj: $ btcdeb '[304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a2efaaf96d022031d3646e4c11f8dbd02369e530f483ab5f60c9e2983f1828ec1478e250b617a401 03f639aa08d34e668cbce9d6ddf9a981e9f00b7715
f0e8dce6550755d55b8e2dc0]' 'OP_DUP OP_HASH160 62ae5de53fd6e18d94911353a4627b2ee8936228 OP_EQUALVERIFY OP_CHECKSIG'
btcdeb 5.0.24 -- type `btcdeb -h` for start up options
LOG: signing segwit taproot
notice: btcdeb has gotten quieter; use --verbose if necessary (this message is temporary)
warning: opcode-like string was not an opcode: DUP OP_HASH160 62ae5de53fd6e18d94911353a4627b2ee8936228 OP_EQUALVERIFY OP_CHECKSIG
2 op script loaded. type `help` for usage information
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a... | 4f505f445550204f505f484153483136302036326165356465353366643653...
03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0 |
#0000 304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a2efaaf96d022031d3646e4c11f8dbd02369e530f483ab5f60c9e2983f1828ec1478e250b617a401
btcdeb> step
                <> PUSH stack 304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a2efaaf96d022031d3646e4c11f8dbd02369e530f483ab5f60c9e2983f1828ec1478e250b617a401
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0 | 304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a...
                                                                        | 4f505f445550204f505f484153483136302036326165356465353366643653...
#0001 03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0
btcdeb> step
                <> PUSH stack 03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
                                                                        | 03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0
                                                                        | 304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a...
                                                                        | 4f505f445550204f505f484153483136302036326165356465353366643653...
btcdeb> step
script                                                                  |                                                                  stack
------------------------------------------------------------------------+------------------------------------------------------------------
                                                                        | 03f639aa08d34e668cbce9d6ddf9a981e9f00b7715f0e8dce6550755d55b8e2dc0
                                                                        | 304402207c890728bb9436bfbf5d785b4d6cdeab982bbd0ad42f8e19fa3741a...
                                                                        | 4f505f445550204f505f484153483136302036326165356465353366643653...
btcdeb> step
at end of script
btcdeb>
```

We have encapsulated all the possible step commands in the above mentioned image.

Step-5: Final Summary for the entire code

```
Transaction Sizes
A->B size: 247
A->B vsize: 166
A->B weight: 661
B->C size: 247
B->C vsize: 166
B->C weight: 661
```

Summary:

- SegWit reduces transaction size because the witness data is discounted when computing the virtual size (vsize), which is calculated as weight / 4.

We have successfully implemented the requirements and also saved the transactions as json files into the code base which represent the

transactions being done in code. We have also saved json files for the btcdeb (bitcoin debugger) commands into the codebase as well. The requirements showing funding, transactions from A to B and then B to C and script verification and btcdeb are all done.

```
varshith@varshith:~/Desktop/BlockChian/CS216-Malicious_Nodes-Bitcoin_Transaction_Lab$ python3 src/part2_segwit.py
SegWit P2SH-P2WPKH Transactions
Connected to: regtest
Address A: 2MxgVrYi8coPBUmaNwP6heAJyCydBfZkiey
Address B: 2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs
Address C: 2NDzbuBLSwnQQV7F9MDXaXXr25g7BLWKSiY

Mining 101 blocks to A

Creating Transaction A -> B (2 BTC)
TXID: ec59ddc4f77ef85ec6925e4745976d56ad97270db9f97dffa5ba470dcc326474

Creating Transaction B -> C (1 BTC)
TXID: ded13b0b763b5b62c15d9448ab850de8c8cad0be24d4fd2eacdc7b919809fe62

Generating btcdeb stack-trace command

Witness Stack:
Signature: 304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce622275fe76b36c90a23b4c8f6b77ff3679b38b03d
efcbb801
Public Key: 029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473

Run this command for btcdeb debugging:

btcdeb '[304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce622275fe76b36c90a23b4c8f6b77ff3679b38b03def
cbb801 029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473]' 'OP_DUP OP_HASH160 444a86b3439c8b23c9b53a22fd19345de2dc53ae OP_EQUALV
ERIFY OP_CHECKSIG'

Transaction Sizes
A->B size: 247
A->B vsize: 166
A->B weight: 661
B->C size: 247
B->C vsize: 166
B->C weight: 661
```

The above screenshot shows a complete run of the code shows the entire output given by the code and shows transaction creation, btcdeb command generation and also the final stages of the transactions.

PART -3

Analysis and Comparison of Legacy (P2PKH) and SegWit (P2SH-P2WPKH) Transactions

Transaction Size Comparison

- The transactions we have used are at some stage of development and we have made a few changes and have rerun the code which may cause the data in the report here not similar to the transaction data in the repository but the code structure of differences and comparisons remain the same.

The transactions generated in **Part-1 (Legacy P2PKH)** and **Part-2 (SegWit P2SH-P2WPKH)** were analyzed using `decoderawtransaction` to extract their size related metrics.

Part 1 – Legacy P2PKH Transaction Sizes

Transaction A->B:

```
"version": 2,
"size": 225,
"vsize": 225,
"weight": 900,
"locktime": 0,
```

Transaction B->C:

```
"version": 2,
"size": 225,
"vsize": 225,
"weight": 900,
"locktime": 0,
```

Part 2 – SegWit P2SH-P2WPKH Transaction Sizes

Transaction A->B:

```
"version": 2,
"size": 247,
"vsize": 166,
"weight": 661,
"locktime": 0,
```

Transaction B->C:


```
"version": 2,
"size": 247,
"vsize": 166,
"weight": 661,
"locktime": 0,
```


Side-by-Side Comparison Table

Transaction Type	Size (bytes)	Virtual Size (vbytes)	Weight Units
Legacy P2PKH	225 bytes	225 vbytes	900
SegWit P2SH-P2WPKH	247 bytes	166 vbytes	661

Although the total byte size of the SegWit transaction appears slightly larger due to additional structural fields (marker, flag, witness stack), the **virtual size (vsize)** and **weight units** are significantly lower. This occurs because the **witness data is discounted when calculating transaction weight** — witness bytes count as 1 weight unit each, whereas non-witness bytes count as 4 weight units each.

Script Structure Comparison

Legacy P2PKH Transaction Scripts

In Legacy transactions, both the unlocking and locking scripts are executed together in the same validation context. The scripts are concatenated and run sequentially on the stack.

Locking Script (scriptPubKey / Challenge)

```
"vout": [
  {
    "value": "2.00000000",
    "n": 0,
    "scriptPubKey": {
      "asm": "OP_DUP OP_HASH160 257fffb3b660aa35c23711be39a940232d2f76d7 OP_EQUALVERIFY OP_CHECKSIG",
      "desc": "addr(miwEdfdJmxbssMyUW9JnF3fJH3pE34kmPB)#teenve37",
      "hex": "76a914257fffb3b660aa35c23711be39a940232d2f76d788ac",
      "address": "miwEdfdJmxbssMyUW9JnF3fJH3pE34kmPB",
      "type": "pubkeyhash"
    }
  },
```


Unlocking Script (scriptSig / Response


```
        "txid": "638f1a2fd8b4e9e9bb16a2658f83b9d599f9f2b5638b78d4e7c125231f78181d",
        "vout": 0,
        "scriptSig": {
          "asm": "30440220236c19017fc62e23325973e06e8ecb6ef30135b58f1337f75e6c-
5e9892b225b902207b4cf902f5fa9fd34c2cb26a385e6c62c916f8452ba7259af1f0c9654898d930[ALL]
0366f8b6c59c902b3ffe223353079af5de43b6c478bf7fb9a75679b55599fffb43",
          "hex": "4730440220236c19017fc62e23325973e06e8ecb6ef30135b58f1337f75e-
6c5e9892b225b902207b4cf902f5fa9fd34c2cb26a385e6c62c916f8452ba7259af1f0c9654898d9-
3001210366f8b6c59c902b3ffe223353079af5de43b6c478bf7fb9a75679b55599fffb43"
        },
        "sequence": 4294967293
      }
```


The **challenge script** uses `OP_DUP OP_HASH160` to duplicate the provided public key and hash it with HASH160. `OP_EQUALVERIFY` checks that the computed hash matches the stored public key hash. Finally `OP_CHECKSIG` verifies the cryptographic signature, proving ownership of the funds.

SegWit P2SH-P2WPKH Transaction Scripts

In SegWit transactions, the signature and public key are separated from the main transaction data and stored in the **witness field**. The `scriptSig` only holds the SegWit redeem script reference, keeping the main transaction body compact.

Locking Script (scriptPubKey / Challenge)


```
{
  "value": "2.00000000",
  "n": 0,
  "scriptPubKey": {
    "asm": "OP_HASH160 045cd58f134b43d83b9c19ea231fb23df2fc98b5 OP_EQUAL",
    "desc": "addr(2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs)#k7k62l4z",
    "hex": "a914045cd58f134b43d83b9c19ea231fb23df2fc98b587",
    "address": "2MseHuHWv2HrT5vuQQcpSyd8voh4cvbjErs",
    "type": "scripthash"
  }
},
```

scriptSig (Redeem Script Reference) and Witness Stack


```
{
  "txid": "ec59ddc4f77ef85ec6925e4745976d56ad97270db9f97dffa5ba470dcc326474",
  "vout": 0,
  "scriptSig": {
    "asm": "0014444a86b3439c8b23c9b53a22fd19345de2dc53ae",
    "hex": "160014444a86b3439c8b23c9b53a22fd19345de2dc53ae"
  },
  "txinwitness": [
    "304402200183d0735b3a9d25212277c37e9cf3abed302b6fd28b57e157689456e254477602200e3814487d2e8cdce622275fe76b36c90a23b4c8f6b77ff3679b38b03de
    fcbb801",
    "029d3c3e1515ba0c78d7828d70826e8d4dcb47f6bc5586c35223a6a11a4b2b7473"
  ],
  "sequence": 4294967293
}
```


The redeem script in the scriptSig references the SegWit witness program, and the actual unlocking data is provided through the witness stack. During validation, the public key is hashed and compared with the stored hash, and the signature is verified using `OP_CHECKSIG`.

Script Structure Comparison Table

Feature	Legacy (P2PKH)	SegWit (P2SH-P2WPKH)
Unlocking Data Location	Inside scriptSig	Moved to the witness stack
scriptSig Content	<Signature> <Public Key>	Redeem Script reference only (e.g., 0014<PubkeyHash>)
Execution Flow	Concatenates scriptSig and scriptPubKey, then executes.	Evaluates scriptSig, then evaluates the witness data against the referenced program.
Malleability	Susceptible (signatures can be altered changing the TXID).	Fixed (signatures are outside the TXID calculation).

Why SegWit Transactions are More Efficient

Segregated Witness improves transaction efficiency by separating signature data from the main transaction structure. The weight formula for a SegWit transaction is:

```
Weight  =  (non-witness bytes  x  4)  +  (witness bytes  x  1)

Legacy  P2PKH:        225 bytes x 4  =  900  weight units
                      vSize = ceil(900 / 4) = 225 vbytes

SegWit  P2SH-P2WPKH:
  Non-witness part: ~110 bytes x 4  =  440
  Witness part:     ~107 bytes x 1  =  107
  Remaining fields: ~28  bytes x 4  =  114
  Total weight                      =  661  weight units
  vSize = ceil(661 / 4)             =  166  vbytes   ← 26% smaller
```

Key advantages of SegWit:

- **Reduced virtual transaction size (vsize)** — 166 vs 225 vbytes (~26% reduction), allowing more transactions to fit within a block.
- **Lower effective transaction fees** — fees are calculated based on vbytes rather than raw bytes, so SegWit users pay proportionally less.
- **Elimination of transaction malleability** — signature data is moved outside the transaction ID calculation, improving reliability for higher-layer protocols such as the Lightning Network. Furthermore, the witness program includes a version byte, paving the way for easier future network upgrades (like Taproot).
- **Improved block capacity** — the effective block capacity increases from 1 MB to up to 4 MB in weight units (~1.7–2 MB in practice), enabling the network to process more transactions per block.
- **Backward-compatible deployment** — P2SH-P2WPKH wraps the SegWit witness program inside a familiar P2SH envelope, ensuring compatibility with older wallets that do not natively support SegWit

Conclusion

Through the implementation of both transaction types in regtest mode, we observed that while SegWit transactions contain additional structural components, their **virtual size and weight are significantly smaller than**

Legacy transactions. This demonstrates how SegWit optimises transaction efficiency while maintaining the same security guarantees through cryptographic signature validation.

The P2PKH script model executes both the locking and unlocking scripts inline, whereas the P2SH-P2WPKH model separates the witness data from the transaction identifier, enabling fee savings and improved scalability without breaking backward compatibility.